School of Future Tech

Case Study Report

on

Digital Wallet Simulation (Encapsulation-Based)

by

Manan JAIN

150096725166

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Digital payments have replaced physical cash in most student and professional environments. This project DigiWallet is a functional simulation of a fintech application. It manages a user's digital balance through a secure interface, focusing on the real-world logic of adding funds (Credits) and making payments (Debits). The goal was to create a smooth, interactive experience where financial transactions are processed instantly without page reloads.

2. Problem Statement / Case Background (Abstract)

Background

Basic web applications often lack proper data protection. If a user's wallet balance is accessible by any script, it can be manipulated easily. Furthermore, traditional apps often fail to provide instant feedback for successes or failures, making them feel unresponsive.

Abstract

The DigiWallet system addresses these issues by using JavaScript **Encapsulation**. By housing all financial logic within a Wallet class, the balance is protected from external tampering. The system allows users to initialize a wallet with their name, perform validated top-ups, and make payments that are checked against the available balance. A chronological ledger (transaction history) is maintained in the background and displayed dynamically on the UI.

3. Case Study Design

The project is built as a single-page interactive dashboard.

- **Initialization:** The user begins at a login screen. Upon entering their name, the JavaScript creates a new instance of the Wallet class.
- **The Wallet Logic:** The core of the app is the Wallet function/class. It manages two private variables: _balanceand _transactions.
- **Transaction Workflow:** * **Top-Up:** Adds to the balance and records a Top Up object.
    - **Payment:** First validates if amount <= _balance. If not, it rejects the transaction and triggers a Shake animation on the UI.
- **DOM Syncing:** A dedicated updateUI() function runs after every action to ensure the balance text and history list match the internal data.

4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study

Logic & Algorithms:

- **Validation Algorithm:** Before any math happens, the system checks isNaN(amount) and amount <= 0. For payments it specifically uses a conditional check to prevent negative balance (Overdraft).
- **Encapsulation:** I used function-level scoping for _balance and _transactions. Since these are not attached to this they cannot be

modified directly from the browser console; only the topUp and makePayment methods can touch them.

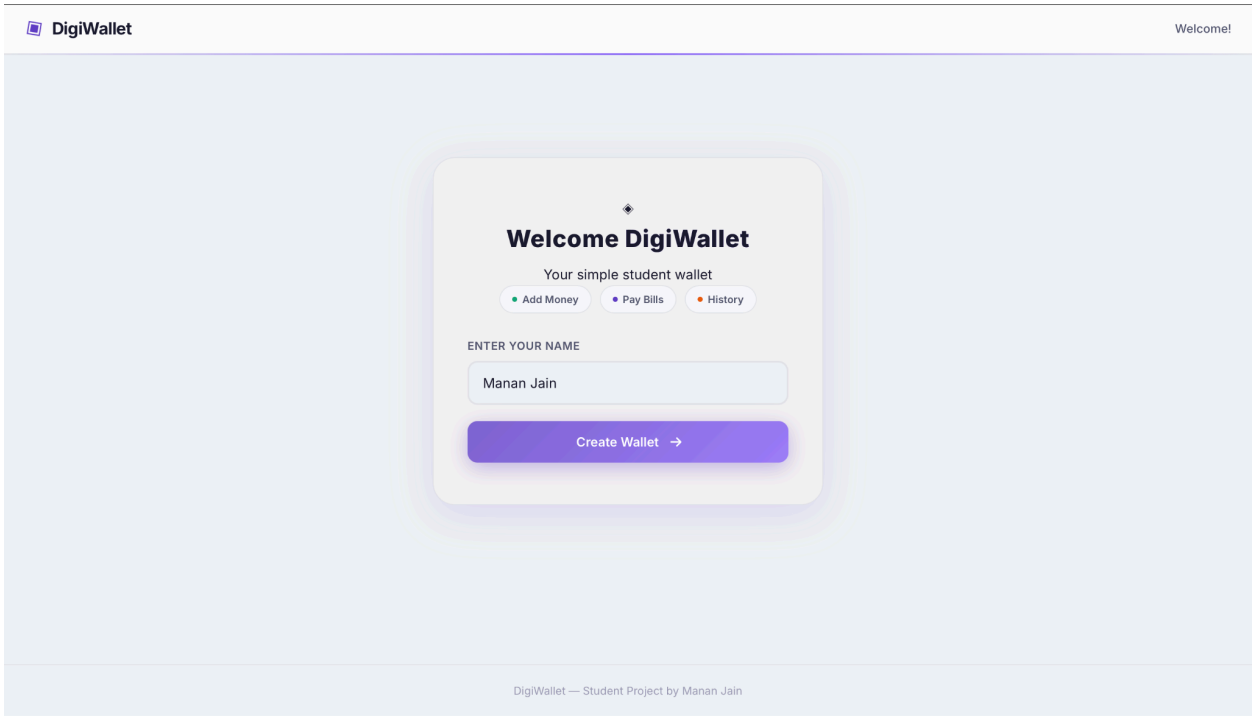
- **Array Management:** Transaction objects are stored in an array. To show the most recent activity first, the renderHistory() function uses the [...historyData].reverse() method.

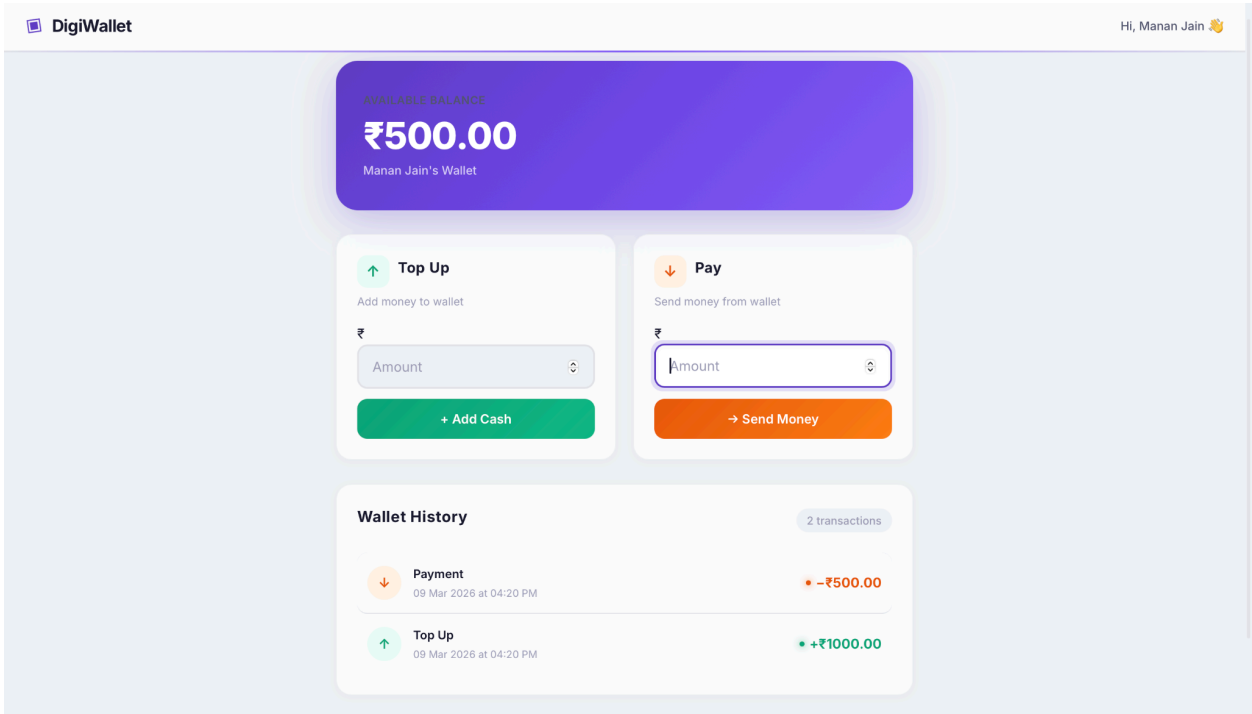
Technology Stack:

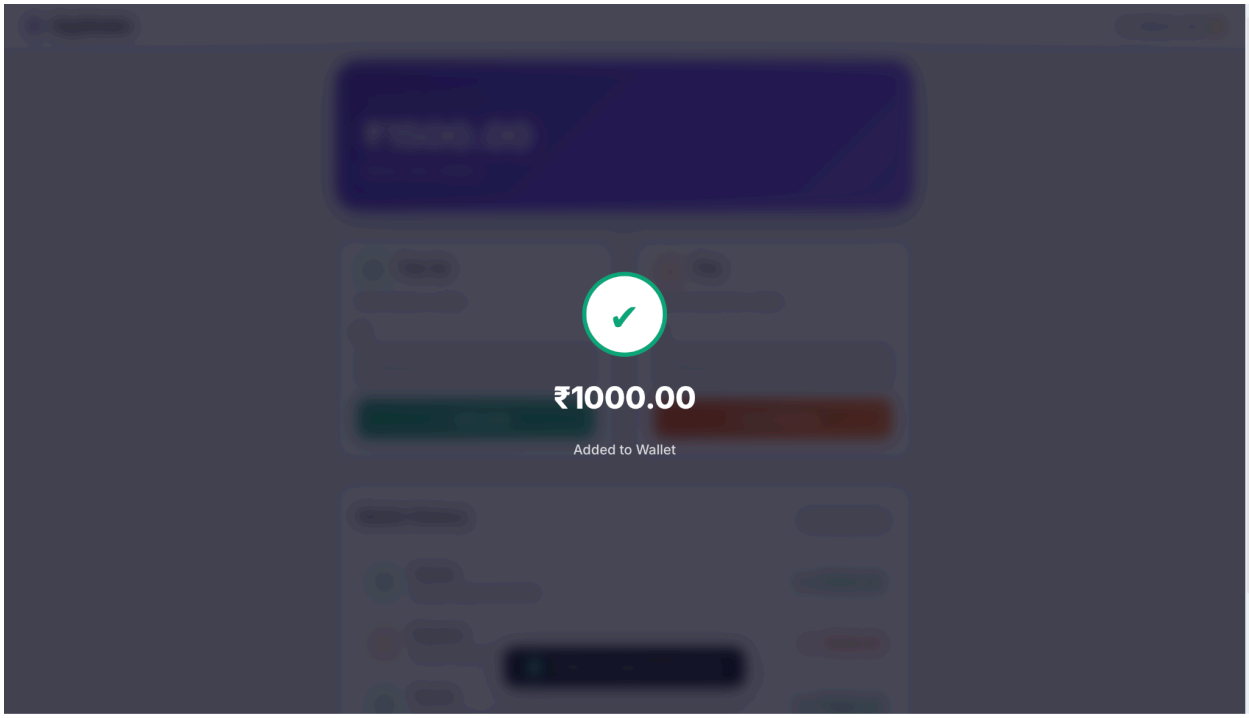
- **HTML5:** Structured with semantic tags like <nav>, <main>, and <section>.
- **CSS3:** Uses Flexbox and CSS Grid for a responsive layout. I also implemented Keyframe Animations(gentleSpin, balancePop) to make the UI feel alive.JavaScript (ES6): Utilizes Classes/Constructor functions Template Literals, and Event Listeners.

5. Problem Statement / Case Study Implementation Details and Snapshots.

Classes: I implemented a Transaction class to standardize how every record is saved, including an automatic timestamp using toLocaleString("en-IN").
UI Feedback: I built a custom Toast Notification and Modal System. If a payment fails the balance card shakes (CSS headShake) and a red error modal appears. If it succeeds, a ripple effect triggers.
Dynamic Rendering: Instead of hardcoding the history, the JavaScript creates <li> elements on the fly and injects them into the ul#history element.


DigiWallet
Welcome!
Welcome DigiWallet
Your simple student wallet
Add Money
Pay Bills
History
ENTER YOUR NAME
Manan Jain
Create Wallet →
DigiWallet — Student Project by Manan Jain


DigiWallet
Hi, Manan Jain 👋
AVAILABLE BALANCE
₹500.00
Manan Jain's Wallet
Top Up
Add money to wallet
₹
Amount
+ Add Cash
Pay
Send money from wallet
₹
Amount
→ Send Money
Wallet History
2 transactions
Payment
09 Mar 2026 at 04:20 PM
−₹500.00
Top Up
09 Mar 2026 at 04:20 PM
+₹1000.00



6. Problem Statement / Case Study Results and Conclusion.

Findings

- **User Experience:** Adding small animations (like the balance-pop) makes the app feel much more premium and responsive.
- **Logic Safety:** Encapsulating the balance ensures that the "Source of Truth" remains accurate regardless of what the user does on the front end.
- **Validation:** Checking for valid numbers (parseFloat) is essential to prevent "NaN" errors in financial calculations.

Conclusion

The DigiWallet project successfully demonstrates how to handle financial data in a web browser. It combines clean UI design with robust JavaScript logic. By using classes and encapsulation, I created a system that is easy to scale and safe to use. This project helped me master the connection between Backend Logic (the Wallet class) and Frontend Display (the DOM).

7. References

MDN Web Docs: JavaScript Classes and Scoping.
Google Fonts: The "Inter" font family.
CSS Tricks: Guide to Flexbox and CSS Grid.
ITM Skills University: Case Study Guidelines for Sem II.